

Junior AI Engineer — Screening Task

Innotechwise Private Limited (Noida) · Futuryntix Group

Welcome. Before we move to an interview, we would like you to attempt a hands-on task. This task is intentionally shaped around the kind of work you would actually do at Innotechwise — building integration-heavy AI services for enterprise customers using a vibe-coding workflow.

About the work you will do here

We build AI-first products for warehouse, supply chain, and compliance workflows. Our junior engineers work directly with the founder on shipping production systems using Claude Code, Cursor, and similar AI-native tools. Day-to-day, the work spans six areas: building **customer solutions**, designing **APIs**, wiring up **integrations** with enterprise systems, applying **LLMs** and broader **AI techniques** meaningfully (not as a buzzword), and doing all of it with **vibe coding** as the default workflow. This task touches all six.

The scenario

You are building a small backend service for **"GreenPack Industries"**, a fictional plastic packaging producer in India. GreenPack must comply with EPR (Extended Producer Responsibility) rules: every month they declare the quantity of plastic they have put into the market, and they must reconcile that declaration against their actual procurement records from their ERP system.

They have asked for a service that does three things: (1) accept their monthly declaration, (2) reconcile it against their ERP feed and produce a human-readable summary, and (3) let their compliance officer ask plain-English questions about EPR rules.

What to build

A Python service (FastAPI recommended, but use what you know) exposing three HTTP endpoints. Choose any storage layer (SQLite, JSON file, in-memory — your call). Choose any LLM (Claude, OpenAI, Gemini, or local Ollama). Choose any vector store for the RAG piece.

Endpoint 1 · POST /submit

Purpose: GreenPack submits their monthly plastic declaration.

Request body example:

```
{
  "producer_id": "GREENPACK-001",
  "month": "2026-04",
  "declared_quantities_kg": {
    "rigid_plastic": 12000,
    "flexible_plastic": 8500,
    "multilayer_plastic": 3200
  }
}
```

- Validate the payload deterministically — required fields present, no negative weights, valid month format. Use Pydantic or your validation library of choice.
- Store the record with a generated record_id and timestamp.
- Return the stored record.
- **This endpoint should not call the LLM.** Validation is a deterministic problem. Knowing when not to use an LLM is part of what we are evaluating.

Endpoint 2 • GET /summary/{producer_id}/{month}

Purpose: Reconcile the declaration against ERP procurement records, then summarize.

- Read the stored declaration from Endpoint 1.
- Read a mock ERP feed — **you create this file**, a CSV or JSON representing GreenPack's actual plastic procurement for that month, broken down by the same categories. Sample structure below.
- Reconcile the two: flag any category where the declared quantity differs by more than 5% from the procured quantity.
- Use an LLM to generate a 3-5 sentence human-readable summary that explains gaps in plain English and recommends an action. **This is structured generation**, not free-form chat — the LLM's job is narrative, not analysis.
- Return a JSON response with both the structured reconciliation result and the LLM-generated narrative.

Sample mock ERP feed (you create this):

```
producer_id,month,category,procured_kg
GREENPACK-001,2026-04,rigid_plastic,11800
GREENPACK-001,2026-04,flexible_plastic,9100
GREENPACK-001,2026-04,multilayer_plastic,3150
```

Endpoint 3 • POST /ask

Purpose: GreenPack's compliance officer asks plain-English questions about EPR rules.

- Accept a JSON body: {"question": "..."}.
- Build a small RAG pipeline over 3-5 documents on EPR / plastic compliance. You can use publicly available sources — CPCB notifications, Wikipedia entries on plastic waste, government circulars, or fabricate small mock policy documents. Cite your sources in the README.
- Return the answer with **citations** identifying which document and section the answer came from.
- If the question cannot be answered from the corpus, return **"I do not know based on the provided documents"**. Do not hallucinate.

Deliverables

- **Public GitHub repository** with code, README, sample data files (your mock ERP feed + your RAG corpus), and a sample curl/script that demonstrates all three endpoints working in sequence.
- **90-second Loom video** covering: (a) a quick demo showing one endpoint working end-to-end, (b) a clip of you using Claude Code / Cursor / Copilot to write or refactor a section of the code — we want to see the AI interaction, not just the output, (c) one architectural trade-off you made and why.
- **README must explicitly state:** which LLM and embedding model you chose and why; which storage and vector store; which AI coding assistant you used and where; one thing you would do differently with another day.

Submission deadline	3 days from receiving this task
How to submit	Reply on the same channel with GitHub link + Loom link
Estimated time	4–6 hours for someone comfortable with FastAPI + an LLM API
Allowed	AI coding assistants (encouraged), tutorials (cite them), open-source libraries
Not allowed	Copy-pasting an existing public repo unchanged. We will check.

What we evaluate

The task is intentionally designed to give us signal across six dimensions. You do not need to ace all six — but a submission that hits at least four well is a strong submission.

Customer Solution
API Design
Integration
LLM Use
AI / Architectural Judgment
Vibe Coding

What you can expect from us

- We review every submission and reply within 3 working days.
- If we move forward, the next step is a 30-minute technical screen, followed by a 45-minute conversation with the founder.
- The role is full-time, Noida-based (hybrid acceptable), CTC band **■3–4 LPA** depending on demonstrated skill.
- You will also be part of business development and sales across the globe, any sales made using your effort will be equally rewarded with a percentage cut in the sales done of the softwares built and sold under the banner of the company with appropriate currency conversion of the currency.
- In-Effect , you can start at 3-4 LPA, but if you are able to scale sales and development for the company, you can earn any amount depending on the products you are handling as sales done in \$ shall be paid %age wise in with currency conversion in INR.
- The work you build here is yours — keep it on your GitHub, reference it in any future interview.

Quick FAQ

Yes — cite it in your README. An honest "I started from this tutorial and adapted X" beats code that looks suspiciously like a Medium article.

Do I need real EPR knowledge?

No. Use any public source as your RAG corpus — CPCB rules, Wikipedia, or fabricate mock policy documents. The point is the RAG pipeline, not the legal content.

Paid APIs or free?

Either works. Free-tier credits are usually enough. To avoid all spend, run a local model with Ollama — that is exactly the workflow we use.

What if I cannot finish in 3 days?

Deadline is fixed.

What if I am not from Noida?

The role is Noida-based (remote). If you are open to relocating, you are welcome to apply — just mention it in your submission message.

Good luck — and reach out if anything in this brief is unclear. We look forward to seeing what you build.

Aman Jain

Founder & Director, Innotechwise Private Limited

Futuryntix Group